

Explain obstacle course LIS

Absolutely. This problem looks hard because it combines **LIS (Longest Increasing Subsequence)** with a small but very important modification: **equal values are allowed**.

The key is to understand the intuition first, and only then the `upper_bound` code.

1. What is the problem asking?

For every position `i`, we need to find:

The longest subsequence from `0...i` that ends at `obstacles[i]` and is **non-decreasing**.

Non-decreasing means:

```
previous <= current
```

So:

1, 2, 2, 5	✓
1, 3, 5	✓
2, 2, 2	✓
3, 2	✗

Also, the obstacles must remain in their original order.

For example:

```
obstacles = [1, 2, 3, 2]
```

At `i = 3`, we need a subsequence ending at `2`.

We cannot use:

```
[1, 2, 3, 2]
```

because:

```
3 > 2 ✗
```

But we can use:

```
[1, 2, 2]
```

So:

```
ans[3] = 3
```

Therefore:

```
[1, 2, 3, 2]
      ↓
answer = [1, 2, 3, 3]
```

2. The obvious/brute-force idea

For every `i`, we could try to find the longest valid subsequence ending at `i`.

Define:

```
dp[i] = longest valid obstacle course ending at i
```

Then:

```
dp[i] = 1 + max(dp[j])
```

where:

```
j < i
obstacles[j] <= obstacles[i]
```

For example:

```
obstacles = [3, 1, 5, 6, 4, 2]
```

For 5:

```
3 <= 5
1 <= 5
```

So we can have:

```
[3,5]
[1,5]
```

Therefore:

```
dp[2] = 2
```

The problem is that checking every previous j gives:

```
 $O(n^2)$ 
```

If n is large, this is too slow.

So we need to somehow find the best previous subsequence **faster than $O(n)$** .

This leads us toward LIS.

3. The important observation

This problem is basically:

```
Longest Increasing Subsequence
```

but with one difference.

Normal LIS requires:

```
a < b
```

This problem requires:

```
a <= b
```

So this is a:

```
Longest Non-Decreasing Subsequence
```

And there is a famous $O(n \log n)$ technique for LIS using a vector often called:

```
tails
```

Your code calls it:

```
lcs
```

although `tails` would be a more descriptive name.

4. What does `lcs` actually mean?

This is the most important part to understand.

Suppose:

```
obstacles = [3, 1, 5, 6, 4, 2]
```

We maintain:

```
vector<int> lcs;
```

Think of:

```
lcs[k]
```

as:

```
The smallest possible ending value of a valid non-decreasing subsequence of length  $k + 1$  found so far.
```

Notice:

It does NOT necessarily store the actual subsequence.

It stores the best possible **tail/end value**.

5. Why do we care about the smallest ending value?

Suppose we have two subsequences of the same length:

```
[1, 5]
[2, 5]
```

Both have length 2.

Which one is more useful for future elements?

Obviously:

```
[1, 5]
```

because smaller ending values give us more opportunities to extend the subsequence.

For example, if the next number is:

```
5
```

both can extend.

But if the next number is:

```
3
```

then:

```
[1, 5] + 3  ✗
[2, 5] + 3  ✗
```

Actually neither works because both end at 5.

But if we instead had:

```
[1, 3]
[2, 4]
```

then 3 can extend:

```
[1, 3] + 3
```

while [2,4] cannot.

So:

For the same subsequence length, a smaller tail is always better.

This is the core intuition behind the LIS algorithm.

6. Now look at your code

Your code is:

```
class Solution {
public:
    vector<int> longestObstacleCourseAtEachPosition(vector<int>& obstacles) {
        int n = obstacles.size();
        vector<int> lcs;
        vector<int> result(n);

        for(int i = 0 ; i < n ; i++){

            int idx = upper_bound(
                lcs.begin(),
                lcs.end(),
                obstacles[i]
            ) - lcs.begin();

            if(idx == lcs.size()){
                lcs.push_back(obstacles[i]);
            }
            else{
                lcs[idx] = obstacles[i];
            }
        }
    }
};
```

```

    }

    result[i] = idx + 1;
}

return result;
}
};

```

Let's break it into three important pieces:

```
upper_bound()
```

then

```
lcs[idx] = obstacles[i]
```

and finally:

```
result[i] = idx + 1
```

7. Why upper_bound?

This is the biggest trick.

For a normal LIS where:

```
a < b
```

we generally use:

```
lower_bound()
```

But here we allow:

```
a <= b
```

Therefore we use:

```
upper_bound()
```

Difference

Suppose:

```
lcs = [1, 2, 2, 4]
```

and current obstacle is:

```
2
```

upper_bound finds:

```
first element strictly greater than 2
```

So:

```
[1, 2, 2, 4]
      ↑
```

It returns index:

```
3
```

because:

```
lcs[3] = 4
```

is the first value greater than 2.

Therefore:

```
idx = 3;
```

8. Why is that exactly what we need?

Because equal values are allowed.

Suppose:

```
lcs = [1, 2, 2]
```

and current value is:

```
2
```

We want to create:

```
[1, 2, 2, 2]
```

So the current 2 should be placed **after the existing 2s**.

That's exactly what:

```
upper_bound()
```

does.

It finds the first value:

```
> current
```

rather than:

```
>= current
```

Therefore it allows duplicates to extend the subsequence.

9. Let's dry run the entire example

Consider:

```
obstacles = [1, 2, 3, 2]
```

Initially:

```
lcs = []
result = []
```

i = 0

Current:

```
obstacles[0] = 1
```

We do:

```
idx = upper_bound(lcs.begin(), lcs.end(), 1)
```

lcs is empty.

So:

```
idx = 0
```

Since:

```
idx == lcs.size()
```

we do:

```
lcs.push_back(1);
```

Now:

```
lcs = [1]
```

And:

```
result[0] = idx + 1
          = 0 + 1
          = 1
```

So:

```
result = [1]
```

10. i = 1

Current:

```
2
```

Current:

```
lcs = [1]
```

Find first value greater than 2.

There isn't one.

So:

```
idx = 1
```

Therefore:

```
lcs.push_back(2);
```

Now:

```
lcs = [1, 2]
```

And:

```
result[1] = 1 + 1 = 2
```

So:

```
result = [1, 2]
```

We have:

```
[1,2]
```

length 2.

11. i = 2

Current:

```
3
```

Current:

```
lcs = [1,2]
```

Find first value greater than 3.

None exists.

So:

```
idx = 2
```

Append:

```
lcs.push_back(3);
```

Now:

```
lcs = [1,2,3]
```

And:

```
result[2] = 2 + 1 = 3
```

So:

```
result = [1,2,3]
```

12. i = 3 — the interesting case

Current:

```
2
```

Current:

```
lcs = [1,2,3]
```

Now:

```
upper_bound(lcs.begin(), lcs.end(), 2)
```

means:

Find the first number strictly greater than 2.

That's:

```
[1, 2, 3]
      ↑
      3
```

So:

```
idx = 2
```

Now:

```
lcs[idx] = obstacles[i];
```

becomes:

```
lcs[2] = 2;
```

So:

```
lcs = [1,2,2]
```

And:

```
result[3] = idx + 1
          = 2 + 1
          = 3
```

Final:

```
result = [1,2,3,3]
```

Exactly what we want.

13. But wait — `lcs = [1,2,2]` doesn't correspond to the actual sequence!

Correct.

This is a very important point.

When we process:

```
[1,2,3,2]
```

we end with:

```
lcs = [1,2,2]
```

But that doesn't mean we actually selected:

```
1,2,2
```

from the array in that exact way?

Actually, in this example it does correspond to a valid sequence, but in general `lcs` is not guaranteed to represent an actual subsequence.

It is a data structure that stores the best possible tail values.

Its purpose is:

```
Help us determine the length of the best subsequence efficiently.
```

That's why the name `lcs` can be slightly misleading.

A better name would be:

```
tails
```

14. Let's understand the replacement

This line:

```
lcs[idx] = obstacles[i];
```

is extremely important.

Why don't we simply append every value?

Because we want to keep the tail as small as possible.

Example:

```
lcs = [1, 3, 5]
```

Suppose current value is:

```
2
```

`upper_bound` finds:

```
3
```

at index `1`.

So:

```
lcs[1] = 2;
```

Now:

```
lcs = [1,2,5]
```

We have effectively improved our possible subsequence of length `2`:

Before:

```
[1,3]
```

After:

```
[1,2]
```

Both have length `2`.

But `[1,2]` is better because its ending value is smaller.

This gives us more opportunities to extend it later.

15. Another dry run: `[3,1,5,6,4,2]`

This example is excellent for understanding the replacement idea.

Start

```
lcs = []
```

```
3
```

```
lcs = [3]  
result = [1]
```

1

First value > 1 is 3.

So:

```
lcs = [1]
```

Notice what happened.

We replaced:

```
[3]
```

with:

```
[1]
```

Both represent a subsequence of length 1.

But ending at 1 is much better than ending at 3.

So:

```
result = [1,1]
```

5

First value > 5 doesn't exist.

Append:

```
lcs = [1,5]
```

Therefore:

```
result = [1,1,2]
```

6

Append:

```
lcs = [1,5,6]
```

Therefore:

```
result = [1,1,2,3]
```

4

Now:

```
lcs = [1,5,6]
```

Find first value:

```
> 4
```

That's 5.

So:

```
idx = 1
```

Replace:

```
lcs[1] = 4
```

Now:

```
lcs = [1,4,6]
```

And:

```
result[4] = 2
```

Why is this useful?

Because:

```
[1,4]
```

is a better length-2 possibility than:

```
[1,5]
```

2

Current:

```
lcs = [1,4,6]
```

Find first value greater than 2.

That's:

```
4
```

at index 1.

Replace:

```
lcs[1] = 2
```

Now:

```
lcs = [1,2,6]
```

And:

```
result[5] = 2
```

Final:

```
[1,1,2,3,2,2]
```

Exactly the expected answer.

16. So what is `idx + 1`?

This is another important observation.

Suppose:

```
idx = 0
```

That means the current value can be the end of a subsequence of length:

```
1
```

Therefore:

```
result[i] = idx + 1;
```

Suppose:

```
idx = 2
```

Then the current value can be the end of a subsequence of length:

```
3
```

because indexes start from 0.

So:

```
idx = 0 → length 1  
idx = 1 → length 2  
idx = 2 → length 3  
idx = 3 → length 4
```

Hence:

```
result[i] = idx + 1;
```

17. The whole intuition in one picture

Think of `lcs` like this:

```
lcs[k] =  
smallest possible ending value  
of a valid subsequence of length k+1
```

For example:

```
lcs = [1, 4, 7]  
      ↓ ↓ ↓  
    len len len  
     1  2  3
```

Meaning:

```
length 1 → smallest possible tail = 1  
length 2 → smallest possible tail = 4  
length 3 → smallest possible tail = 7
```

Now current value is:

```
5
```

We ask:

```
Where can 5 fit?
```

Using:

```
upper_bound(lcs.begin(), lcs.end(), 5)
```

we find first value greater than 5 :

```
[1, 4, 7]  
      ↑  
      7
```

Index:

```
2
```

So 5 can create a subsequence of length:

```
2 + 1 = 3
```

and we improve the tail:

```
[1,4,7]  
      ↓  
[1,4,5]
```

Now a length-3 subsequence can end at 5 instead of 7 .

That's better.

18. Why is `lcs` always sorted?

This is what makes binary search possible.

Suppose:

```
lcs = [1,4,7]
```

The tail for length 2 can't be smaller than the tail for length 1 .

Therefore:

```
lcs[0] <= lcs[1] <= lcs[2] ...
```

So `lcs` is sorted.

That's why we can use:

```
upper_bound()
```

which takes:

```
O(log n)
```

instead of scanning the whole vector.

19. Why does `upper_bound` give the answer for THIS position?

This is the key proof/intuitive connection.

Suppose:

```
lcs = [1,4,7]
```

Current:

```
x = 5
```

`upper_bound(5)` gives index 2.

That means:

```
lcs[1] <= 5
```

but:

```
lcs[2] > 5
```

Therefore we can extend the best possible subsequence of length 2:

```
length 2 + current element  
= length 3
```

So:

```
answer = idx + 1
```

If there were:

```
lcs = [1,4,5,7]
```

and current:

```
x = 5
```

`upper_bound(5)` goes **after the existing 5**:

```
[1,4,5,7]  
  ↑  ↑  
  equal first > 5
```

Actually the first >5 is 7, index 3.

Thus:

```
result = 3 + 1 = 4
```

which correctly allows:

```
..., 5, 5
```

because equal values are permitted.

20. Why not `lower_bound`?

This is one of the most common interview questions.

Suppose:

```
obstacles = [2,2,2]
```

Expected:

```
[1,2,3]
```

Because:

```
2 <= 2 <= 2
```

is valid.

With `upper_bound`:

```
2
→ idx 0
→ lcs = [2]

2
→ first > 2 doesn't exist
→ idx 1
→ lcs = [2,2]

2
→ idx 2
→ lcs = [2,2,2]
```

So:

```
[1,2,3]
```

Perfect.

If we used `lower_bound`, equal elements would replace the existing position instead of extending it, producing the wrong lengths.

Therefore:

```
Strictly increasing → lower_bound
Non-decreasing     → upper_bound
```

This is a very useful rule to remember.

21. Complete algorithm

For every obstacle `x`:

Step 1

Find the first value in `lcs` strictly greater than `x`:

```
int idx = upper_bound(lcs.begin(), lcs.end(), x) - lcs.begin();
```

Step 2

If no such position exists:

```
if(idx == lcs.size())
```

then `x` can extend the longest subsequence.

So:

```
lcs.push_back(x);
```

Step 3

Otherwise replace:

```
lcs[idx] = x;
```

This makes the tail smaller.

Step 4

The length of the best subsequence ending at `x` is:

```
idx + 1
```

So:

```
result[i] = idx + 1;
```

22. Your code with meaningful variable names

I would personally write it like this:

```

class Solution {
public:
    vector<int> longestObstacleCourseAtEachPosition(vector<int>& obstacles) {

        int n = obstacles.size();

        vector<int> tails;
        vector<int> ans(n);

        for (int i = 0; i < n; i++) {

            int idx = upper_bound(
                tails.begin(),
                tails.end(),
                obstacles[i]
            ) - tails.begin();

            if (idx == tails.size()) {
                tails.push_back(obstacles[i]);
            }
            else {
                tails[idx] = obstacles[i];
            }

            ans[i] = idx + 1;
        }

        return ans;
    }
};

```

`tails` is better than `lcs` because it tells you what the vector represents.

23. Complexity

For every element we perform:

```
upper_bound()
```

which takes:

```
O(log n)
```

And we do this for n elements.

Therefore:

```
Time Complexity = O(n log n)
```

The `tails` vector can contain at most n elements:

```
Space Complexity = O(n)
```

24. The intuition you should remember for interviews

Don't try to memorize the code.

Remember these **4 observations**:

Observation 1

The problem asks for a longest subsequence where:

```
a[i] <= a[i+1]
```

So it's a:

```
Longest Non-Decreasing Subsequence
```

Observation 2

The normal DP solution is:

```
dp[i] = 1 + best dp[j]
```

for:

```
j < i
obstacles[j] <= obstacles[i]
```

But that's:

```
O(n2)
```

So we need optimization.

Observation 3

For every subsequence length, keeping the **smallest possible ending value** is best.

Therefore we maintain:

```
tails[k] = smallest tail possible
           for a subsequence of length k+1
```

Observation 4

Because equal values are allowed, we need the first value:

```
> current
```

Therefore:

```
upper_bound()
```

not:

```
lower_bound()
```

And its position tells us the answer:

```
answer[i] = idx + 1;
```

The one-line mental model

When you see this problem, think:

"This is LIS, but duplicates are allowed → maintain minimum tails → use `upper_bound` → **position + 1 is the answer.**"

That's the entire trick behind the solution.